Synthetic QA Dataset Generation via Graph-Grounded Multi-Step Reasoning

Implementation, Design Rationale, and Extension Guide

Technical Implementation Report

April 2026

Contents

1 Pipeline Overview **3**
- 1.1 Core Idea . . . 3
- 1.2 Stage-by-Stage Description . . . 3
- 1.3 Quick Start . . . 4
- 1.4 Key File Locations . . . 5

2 Source Papers and Scope **5**

3 Architecture and Data Flow **5**

4 Data Models **6**

5 LLM Client **7**
- 5.1 Design . . . 7
- 5.2 JSON Parsing (`_parse_json_response`) . . . 7
- 5.3 Adding a Provider . . . 7
- 5.4 Switching to a Local Model . . . 7

6 Tool Ecosystem **8**
- 6.1 Interface . . . 8
- 6.2 Registry and Dispatch . . . 8
- 6.3 Current Tools . . . 9
- 6.4 Adding a Tool . . . 9

7 Knowledge Graph Construction **10**
- 7.1 Atomic Fact Model . . . 10
- 7.2 Build Process (`build_from_seeds`) . . . 10
- 7.3 Search→Visit Chain (`_lookup_and_atomise_search`) . . . 11
- 7.4 Isomorphic Table Extraction (`TableTool`) . . . 11
- 7.5 Intra-Group Edges . . . 11
- 7.6 Inter-Group Edge Discovery (`_discover_edges`) . . . 12
- 7.7 Cross-Topic Edge Discovery . . . 12

8 Knowledge Graph Data Structure **12**

9 Subgraph Sampling **13**
9.1 Random Walk Sampling . . . 13
9.2 Bridging Sampling . . . 13
9.3 Connectivity Guarantee (`_ensure_connected`) . . . 14

10 QA Generation from Subgraph **14**

11 Uncertainty Injection **14**
11.1 Adding an Uncertainty Operation . . . 15

12 Graph Complexity Escalation **15**
12.1 Single Escalation Round . . . 15
12.2 Five Escalation Strategies . . . 17
12.3 Adding an Escalation Strategy . . . 17

13 Continuous Complexity Control **17**
13.1 Design: Generative Parameter Scaling vs Post-Hoc Filtering . . . 17
13.2 Parameter Scaling . . . 18
13.3 Why esc_rounds is Fixed at 1 . . . 18
13.4 Empirical Results . . . 18

14 Quality Control **18**
14.1 Pipeline . . . 19
14.2 Complexity-Aware Judge . . . 19
14.3 Deduplication Implementation . . . 19

15 Trajectory Capture **20**

16 Batch Generation Loop **20**
16.1 Async Architecture . . . 20
16.2 No Early Stopping . . . 20
16.3 Output Files . . . 20

17 Configuration Reference **20**
17.1 `configs/llm_config.yaml` . . . 20
17.2 `configs/tool_config.yaml` . . . 21
17.3 `configs/env.sh` . . . 21

18 Running and Extending the Pipeline **21**
18.1 Build or Rebuild the KG . . . 21
18.2 Generate QA Pairs . . . 22
18.3 Specialising the Pipeline for a Domain . . . 22
18.4 Modifying Prompts . . . 22

19 Visualiser **22**
19.1 Server (`ui/serve.py`) . . . 22
19.2 Viewer (`ui/viewer.html`) . . . 23

20 Known Issues and Fix Locations **23**

1 Pipeline Overview

1.1 Core Idea

Rather than prompting an LLM to generate hard questions directly (which produces unverifiable claims), the pipeline grounds every question in externally retrieved facts stored in a knowledge graph. Every node in the graph is a single atomic fact traceable to one tool call. Questions are written by sampling a subgraph and asking the LLM to connect its nodes — so the answer is verifiable against the graph, which is in turn verifiable against the retrieved sources. Topological properties of the sampled subgraph (diameter, cross-cluster span) directly control question complexity. The LLM's job is to navigate the graph and synthesize questions from it, not to invent facts.

1.2 Stage-by-Stage Description

Knowledge graph construction (Section 7, 8). The KG is built once from a list of seed topics and reused across all generation runs. For each seed, three tools are called: `search` (triggers a full search→page-visit chain), `wikipedia`, and `scholar`. Each tool result is atomised by an LLM into the smallest possible independent facts, each stored as a typed node (`Entity`) with `tool_source` and `tool_query` fields. Facts from the same lookup are connected with `same_topic` edges; facts from different lookups within the same round are cross-linked by an LLM that identifies typed semantic relations (supports, causes, elaborates, etc.). After all seeds are processed, a second cross-topic pass links representative nodes from *different* seed topics — without this, each seed forms an isolated clique and cross-domain questions are impossible. With 14 seeds, this produces 205 entity nodes and 1,106 relations in a single connected component.

Tool ecosystem (Section 6). All tools implement `BaseTool` (`async execute(**kwargs) → dict`) and are registered with `ToolRegistry`. The registry dispatches tool calls from ReAct agents by name (`TOOL_CALL: tool_name | query`), normalising the generic `query` string into tool-specific kwargs. Two tool categories exist: *retrieval tools* (`search`, `visit`, `wikipedia`, `scholar`, `table`) used during KG construction; *computation/reasoning tools* (`calculator`, `python`, `unit_converter`, `timeline`, `comparison`) reserved for the QC advanced solver. Adding a new tool requires only subclassing `BaseTool` and calling `registry.register()`.

Subgraph sampling (Section 9). A `SubgraphSampler` extracts a connected subset of the KG to serve as the knowledge context for one QA pair. Two strategies alternate per batch: *random walk* (starts from a random node, walks `walk_length` undirected steps, deduplicates) and *bridging* (picks `n_components` random seed nodes from different graph regions, collects their 2-hop neighborhoods, stitches them together). Random walk produces locally clustered subgraphs; bridging forces cross-domain span. If the induced subgraph is disconnected, `_ensure_connected` inserts shortest-path intermediate nodes to guarantee connectivity. `walk_length` is the primary dial for subgraph diameter; `n_components` is the primary dial for cross-domain span. Both are scaled by `-complexity`.

QA generation (Section 10). Given a subgraph, the LLM receives a formatted description of all its entities (with values and tool sources) and all relations between them. The prompt (`prompts/graph_qa_generator.txt`) requires the output question to connect at least 3 entities and have a definitive verifiable answer that cannot be inferred from any single entity alone. Output: `QAPair` with `source_chunk_ids` pointing to the entity IDs used.

Uncertainty injection (Section 11). Before escalation, one (configurable: up to six) atomic reframing operation is applied to the question *within the existing subgraph* — no new entities are

added. Operations: entity anonymisation, temporal constraint addition, attribute substitution, condition injection, aggregation requirement, indirection. Each operation is a single LLM call that modifies the question and updates the answer accordingly. Sequential composition: each operation's output is the next operation's input.

Graph escalation (Section 12). The question is rewritten to span additional KG nodes beyond the original subgraph. The escalator finds 1-hop neighbor entities not already in the subgraph (sorted by degree, capped at 12 candidates), then runs a ReAct loop: the LLM sees the current Q/A, the covered entities, the candidate entities, and the tool list. It may call tools to verify candidate facts, then emits a JSON response with the rewritten question, new answer, and which candidate entities it incorporated. The entity IDs are merged into `source_chunk_ids` with connector nodes to maintain connectivity. Five strategies (`hop_extension`, `subgraph_bridging`, `multi_entity_aggregation`, `constraint_chain`, `inverse_reasoning`) are described in the escalation prompt; the LLM selects which to apply. Strategy weights shift toward bridging/aggregation at high complexity values.

Quality control (Section 14). Three sequential filters: (1) *base solver* (LLM, no tools) — if it answers correctly, the question is `TOO_EASY`; (2) *advanced solver* (LLM, full tool access, ReAct loop up to $8 + 4c$ calls) — if it fails, `TOO_HARD`; (3) *semantic deduplication* (cosine similarity of sentence-transformer embeddings, threshold 0.85) — if too similar to a prior PASS, `DUPLICATE`. Base solver is always judged strictly. Advanced solver is judged with complexity-aware leniency: `lenient_paraphrase` at low cx (small subgraphs produce phrasing-divergent answers), `strict` at mid cx, `lenient_partial` at high cx (multi-hop answers may be incomplete but core claim correct).

Trajectory capture (Section 15). For every PASS, the advanced solver's full ReAct chain (thought → tool_call → observation → ... → answer) is recorded as a `Trajectory` and saved to `passing_trajectories.jsonl`. This is the training signal for fine-tuning tool-augmented reasoning models.

Complexity control (Section 13). `-complexity` $c \in [0, 1]$ linearly scales sampling parameters: `walk_length` $10 \to 30$, `bridge_components` $2 \to 4$, `walk_min/max`, solver tool budget. It does not post-filter; it controls the generative process so that every generated question is at the target difficulty. Omitting the flag runs the original fixed pipeline.

1.3 Quick Start

```
pip install openai anthropic sentence-transformers networkx pydantic pyyaml \
            beautifulsoup4 aiohttp requests duckduckgo-search

source configs/env.sh   # sets OPENAI_API_KEY, SEMANTIC_SCHOLAR_API_KEY

# Build the KG once (~30 min, GPT-4o)
python scripts/rebuild_kg.py

# Generate QA pairs
python scripts/generate_until_pass.py --batches 5 --batch-size 8
python scripts/generate_until_pass.py --complexity 0.7 --batches 5 --batch-size 8

# Visualiser (optional)
python ui/serve.py   # -> http://localhost:8765/ui/viewer.html
```

1.4 Key File Locations

Path	Purpose
`scripts/generate_until_pass.py`	Main generation loop
`scripts/rebuild_kg.py` / `expand_kg.py`	KG build / incremental expansion
`src/graph/graph_builder.py`	KG construction, cross-topic edges
`src/graph/subgraph_sampler.py`	Random walk + bridging sampling
`src/graph/uncertainty_injector.py`	Atomic question reframings
`src/pipeline/stage2_graph_escalation.py`	ReAct escalation, 5 strategies
`src/pipeline/stage3_quality_control.py`	Solver + judge + dedup
`src/agents/judge_agent.py`	Complexity-aware judge
`src/tools/tool_registry.py`	`BaseTool` interface + dispatch
`src/utils/llm_client.py`	Async LLM client (OpenAI/Anthropic)
`src/utils/data_models.py`	All Pydantic schemas
`configs/llm_config.yaml`	Model, temperature, token budgets per agent
`configs/tool_config.yaml`	Tool API keys and parameters
`prompts/*.txt`	All LLM prompts (Python `.format()` substitution)
`data/output/knowledge_graph.json`	Serialised KG
`data/output/passing_qas.jsonl`	Final QA pairs (appended across runs)
`data/output/passing_trajectories.jsonl`	Solver ReAct chains for every PASS

2 Source Papers and Scope

This is a reconstruction of two Alibaba/Tongyi Lab pipelines:

- **Track A** — WebFrontier (Qiao et al., arXiv:2509.13309v2): corpus-anchored seed QA pairs iteratively escalated by a tool-augmented ReAct agent.
- **Track B** — Tongyi DeepResearch (arXiv:2510.24701v2): entity-anchored knowledge graph, subgraph sampling for question generation, uncertainty injection, graph escalation.

Track B is the primary implementation (∼4,500 lines, fully functional). Track A is implemented in `scripts/run_track_a.py` but less tested. All analysis below is Track B unless specified.

3 Architecture and Data Flow

Listing 1: Track B pipeline (generate_until_pass.py)

```
# One batch:
# 1. Sample 2*batch_size subgraphs in parallel (Semaphore=8)
#    - odd idx:  sample_bridging(n_components, per_comp)
#    - even idx: sample_by_random_walk(walk_min, walk_max, walk_length)
#    take first batch_size successes
# 2. Uncertainty injection: asyncio.gather all QAs
# 3. Graph escalation: escalate_batch(rounds, max_concurrent=6)
# 4. QC: run_qc_batch(max_concurrent=6)
#    - base_solver.solve_base -> judge(strict)  => TOO_EASY
#    - advanced_solver.solve_advanced -> judge(cx-aware) => TOO_HARD
#    - similarity_scorer.is_duplicate(0.85) => DUPLICATE
#    - else: PASS -> append to passing_qas.jsonl + ui_data.json
```

KG is built once. It requires ∼30–60 min with GPT-4o for 14 seeds. All downstream stages load from `data/output/knowledge_graph.json`. Rebuilding invalidates all QA checkpoints.

Checkpointing. The orchestrator (`src/pipeline/orchestrator.py`) saves each stage as JSONL to `data/checkpoints/{stage}.jsonl`. Stage result is loaded from checkpoint if file exists, skipping the stage. To re-run a stage, delete its checkpoint file.

4 Data Models

All inter-stage data is Pydantic v2 (`src/utils/data_models.py`). Models are serialised via `model_dump()` / `model_validate_json()`.

Listing 2: Core data models

```python
class Entity(BaseModel):
    id: str           # uuid4[:8], generated on construction
    name: str         # short label (< 10 words)
    entity_type: str  # fact|quantity|date|definition|person|place|event
    attributes: dict  # {"value": <fact sentence>, "key_terms": [...]}
    tool_source: str  # which tool produced this node
    tool_query: str   # exact query sent to the tool
    snippet: str      # raw text from tool result

class Relation(BaseModel):
    source_id: str
    target_id: str
    relation_type: str  # supports|elaborates|quantifies|causes|
                        # contradicts|temporal|same_topic
    evidence: str = ""

class QAPair(BaseModel):
    id: str
    question: str
    answer: str
    difficulty: QADifficulty      # seed|escalated_1|2|3|final
    source_chunk_ids: list[str]   # KG entity IDs in this QA's subgraph
    source_subgraph_id: str
    escalation_history: list[str]# question text at each prior escalation step
    required_tools: list[ToolType]
    metadata: dict                # strategy, complexity_level, entities_incorporated

class TrajectoryStep(BaseModel):
    role: str             # "assistant" (thought/tool_call) | "user" (observation)
    content: str
    tool: Optional[str]
    tool_query: Optional[str]
    observation: Optional[str]

class Trajectory(BaseModel):
    qa_id: str
    mode: str = "react"  # "react" | "escalation"
    steps: list[TrajectoryStep]
    final_answer: str
    tool_calls_used: list[str]
    created_at: str
```

Why `source_chunk_ids` instead of embedding the subgraph? IDs are cheap to store and sufficient to reconstruct the full subgraph description from the KG at any point. Embedding

the full subgraph text in every QAPair would bloat JSONL by 10×.

Why `escalation_history` as a list of strings? The history stores only the question text, not the full QAPair. This is sufficient for the UI's escalation chain view and for analysing how questions evolved, without the memory cost of recursive object nesting.

5 LLM Client

File: `src/utils/llm_client.py`

5.1 Design

`LLMClient` is a thin async wrapper over OpenAI and Anthropic SDKs with:

- Provider dispatch via `_call_provider(provider, model, messages, params)`
- Per-agent call params loaded from `configs/llm_config.yaml` by `params_key`
- Exponential backoff retry: 3 retries at delays $[2, 4, 8]$ seconds
- JSON extraction via `complete_json(messages, schema: type[BaseModel])`

5.2 JSON Parsing (`_parse_json_response`)

Three-pass extraction strategy, tried in order:

1. `schema.model_validate_json(raw)` — direct parse
2. Regex extract from markdown code block: ```(?:json)?\s*\n?(.*?)\n?\s*```
3. Regex extract first JSON object: `\{.*\}` (DOTALL)

Why regex instead of guided decoding? The pipeline targets OpenAI-compatible APIs where structured output (`response_format`) is available but not used. Regex is provider-agnostic and works with any OpenAI-compatible endpoint (including local vLLM). The failure rate for well-formed models (GPT-4o) is <1%. For local models that emit think-tokens (e.g. Qwen3), the regex will fail on `<think>...</think>` blocks — fix by stripping think blocks before the three-pass extraction.

5.3 Adding a Provider

Add to `_init_clients`:

```
if "myprovider" in providers:
    from myprovider_sdk import AsyncClient
    self._clients["myprovider"] = AsyncClient(
        api_key=providers["myprovider"]["api_key"]
    )
```

Add to `_call_provider`:

```
elif provider == "myprovider":
    return await self._call_myprovider(model, messages, params)
```

Add `_call_myprovider` following the same pattern as `_call_openai`: extract system messages, call API, return `str`. Then set `default_provider: myprovider` in `configs/llm_config.yaml`.

5.4 Switching to a Local Model

In `configs/llm_config.yaml`:

```yaml
providers:
  openai:
    api_key:   "unused"
    base_url:   "http://localhost:8000/v1"  # vLLM endpoint
    default_model:   "Qwen3-30B-A3B"
```

The `openai.AsyncOpenAI` client respects `base_url`. No other code changes required. Two known local-model issues:

1. Qwen3 emits `<think>...</think>` blocks that break `_parse_json_response`. Fix: strip with `re.sub(r"<think>.*?</think>", "", raw, flags=re.DOTALL)` before the three-pass extraction.
2. Long prompts (>30k tokens in graph escalation) cause structured extraction failures. Fix: add `response_format={"type": "json_schema", "json_schema": ...}` to vLLM calls (guided decoding).

6 Tool Ecosystem

6.1 Interface

Every tool extends `BaseTool` (`src/tools/tool_registry.py`):

```python
class BaseTool(ABC):
    name: str          # string key used in TOOL_CALL directives
    description: str   # shown to LLM in prompt via
        get_descriptions_for_prompt()

    @abstractmethod
    async def execute(self, **kwargs: Any) -> dict:
        # Must return {"result": <any>, "cached": bool, ...}
        ...
```

6.2 Registry and Dispatch

`ToolRegistry` holds `_tools: dict[str, BaseTool]`. Registration:

```python
registry.register(MyTool(config))
```

Dispatch normalises a generic `query` kwarg into tool-specific params before calling `tool.execute(**kwargs)`:

```python
# In ToolRegistry.execute():
if "query" in kwargs:
    q = kwargs["query"]
    if tool_name == "python" and "code" not in kwargs:
        kwargs["code"] = q
    elif tool_name == "visit" and "url" not in kwargs:
        if "|" in q:
            url, goal = q.split("|", 1)
            kwargs["url"] = url.strip(); kwargs["goal"] = goal.strip()
        else:
            kwargs["url"] = q
```

Why this normalisation? ReAct agents emit `TOOL_CALL: tool_name | query` as a single string. The registry must be able to dispatch this without requiring agents to know each tool's exact parameter names.

6.3 Current Tools

Tool	Backend	Pipeline role
`search`	DuckDuckGo / Brave fallback	KG construction (triggers search→visit chain)
`visit`	async HTTP + LLM summariser	KG construction (full page content)
`wikipedia`	Wikipedia REST API	KG construction
`scholar`	Semantic Scholar API	KG construction (academic)
`table`	BeautifulSoup HTML parser	KG construction (structured numeric data)
`calculator`	Python `eval` sandbox	QC advanced solver
`unit_converter`	Pint library	QC advanced solver
`timeline`	Date arithmetic	QC advanced solver
`comparison`	LLM self-knowledge	QC advanced solver
`python`	Sandboxed `exec`	QC advanced solver

Why retrieval tools are excluded from QC-only use during KG building: `comparison` uses LLM parametric knowledge, not external retrieval. Allowing it during KG construction would introduce non-retrieved claims into the KG, breaking the traceability guarantee (every KG node = one external tool call). This boundary is enforced by `GraphBuilder._pick_tools()`, which only selects `search`, `wikipedia`, `scholar`.

6.4 Adding a Tool

1. Create `src/tools/mytool.py`:

```python
class MyTool(BaseTool):
    name = "mytool"
    description = "One sentence for LLM. Usage: mytool | <query>"

    def __init__(self, config: dict = {}):
        self.config = config

    async def execute(self, query: str = "", **kwargs) -> dict:
        result = ...  # your retrieval logic
        return {"result": result, "cached": False}
```

2. Register in `scripts/generate_until_pass.py` (or `orchestrator.py`):

```python
from src.tools.mytool import MyTool
registry.register(MyTool(cfg.get("mytool", {})))
```

3. Add config in `configs/tool_config.yaml`:

```yaml
mytool:
  api_key:   ${MY_API_KEY}
  max_results:   5
```

4. The tool is now available to both graph escalation and the advanced solver automatically. The LLM learns to use it from its `description` string — no prompt changes needed.

To restrict a tool to KG construction only: do not register it in the `generate_until_pass.py` registry (which is also used for QC). Register it only in a custom `GraphBuilder` invocation.

To restrict a tool to QC only: register it normally but exclude it from `GraphBuilder._pick_tools()` by not adding it to the `preferred` list there.

Caching. Tools should cache at `data/.cache/{toolname}/{sha256(query)}.json`. The `wikipedia_tool` and `scholar_tool` implement this pattern. Caching is optional but prevents duplicate API calls during KG rebuilds and QC reruns.

7 Knowledge Graph Construction

File: `src/graph/graph_builder.py`, `src/graph/knowledge_graph.py`

7.1 Atomic Fact Model

Design constraint: every KG node represents exactly one indivisible fact from exactly one tool call. The `tool_source` and `tool_query` fields trace each node to its retrieval step. This is not just for provenance display; it is used during cross-topic edge discovery to label which tool produced each group, and by `describe_subgraph` to generate the entity description shown in QA generation prompts.

Why atomic? Composite nodes (e.g. a Wikipedia paragraph) produce ambiguous edges. An edge between a composite node A and composite node B does not specify which fact in A relates to which fact in B. Atomic nodes allow precise edge typing (`quantifies`, `causes`, etc.) and prevent vague "related to" edges that add no reasoning value.

7.2 Build Process (`build_from_seeds`)

Algorithm 1 `GraphBuilder.build_from_seeds`

```
Require: seed_entities S, expansion_rounds R, cross_topic_sample K
frontier ← S; expanded ← ∅; seed_repr_groups ← {}
for round = 1 ... R do
    round_groups ← []
    for topic in frontier \ expanded do
        expanded ← expanded ∪{topic}
        for tool in [search, wikipedia, scholar] do
            entity_ids ← _lookup_and_atomise(tool, topic)
            round_groups.append({tool, topic, entity_ids, seed_topic})
            update seed_repr_groups[seed_topic] if largest group for that seed
        end for
    end for
    for (g_a, g_b) in pairs(round_groups) do
        _discover_edges(g_a, g_b)
    end for
    next_frontier ← key_terms from round_groups entities, [:max_entities_per_expansion]
end for
if cross_topic_edges then
    for each group g_a in seed_repr_groups do
        partners ← random.sample(other seeds, K)
        for g_b in partners where g_b.seed_topic ≠ g_a.seed_topic do
            _discover_edges(g_a, g_b, max_edges=3)
        end for
    end for
end if
```

Per-tool fact caps. `_DEFAULT_TOOL_FACT_CAPS = {search: 8, wikipedia: 4, scholar: 3}`. The search cap is higher because `search` triggers the search→visit chain (fetching full page content), yielding far more distinct facts than the snippet-only fallback. Wikipedia and Scholar return fixed-length snippets/abstracts where diminishing returns kick in early.

Expansion frontier. After each round, `key_terms` are extracted from newly created entity attributes and used as the next frontier, capped at `max_entities_per_expansion=10`. This implements depth-limited expansion: round 1 covers the seeds, round 2 covers closely related terms. The cap prevents exponential growth while still discovering second-order facts.

7.3 Search→Visit Chain (`_lookup_and_atomise_search`)

For `tool_name == "search"` when `visit` is registered:

1. DuckDuckGo search (retry 3× with 3s backoff per attempt, rate-limit mitigation)
2. Filter results to those with both `url` and non-trivial `snippet`
3. Visit top 3 URLs concurrently (`asyncio.gather`), goal: "Extract facts about: {query}"
4. Extract HTML tables from same 3 URLs concurrently via `TableTool`
5. Concatenate: snippets + page text + table text (in that order, total ≤5000 chars to `FACT_EXTRACTION_PROMPT`)
6. Atomise combined text into ≤ 8 `AtomicFact` objects
7. Node `tool_source` is set to `"search+visit"` (not `"search"`) for transparency

Why visit? Snippet text (140–200 chars) produces 1–2 atomic facts per result. Full page text (2000–3000 chars) produces 5–8. The search→visit chain makes `search` the richest knowledge source in the pipeline.

Why concatenate snippets first? If page fetch fails (timeout, JavaScript-rendered page), the snippet is still present. The LLM sees at least some content regardless.

7.4 Isomorphic Table Extraction (`TableTool`)

BeautifulSoup extracts all `<table>` elements from each visited page. Per page: up to 3 tables, up to 20 rows each. Each row is converted to `{header: value, ...}` dicts. Caption is inferred from `<caption>` or the preceding `<h*>` element. Output is formatted as plain text appended to the page content before atomisation.

Why? Numeric and comparative facts (dates, rankings, quantities) are systematically underrepresented in prose. Tables contain exactly this class of facts. The Tongyi paper (§3.4.1) explicitly cites table extraction as a parallel knowledge source for "isomorphic tables".

7.5 Intra-Group Edges

After atomising a single tool lookup, all resulting entity nodes are connected with `same_topic` edges. This guarantees that each lookup's nodes form a connected component:

```python
for i, eid_a in enumerate(entity_ids):
    for eid_b in entity_ids[i + 1:]:
        shared = set(ea.key_terms) & set(eb.key_terms)
        kg.add_relation(Relation(
            source_id=eid_a, target_id=eid_b,
            relation_type="same_topic",
            evidence=(f"Shared terms: {shared}" if shared
                      else f"Same lookup: {tool}('{query}')")
        ))
```

Why all-pairs? A star topology (one hub node connected to all others) would create a higher-degree hub that biases degree-sorted candidate selection during escalation. All-pairs keeps degree distribution uniform within lookup groups.

7.6 Inter-Group Edge Discovery (`_discover_edges`)

`EDGE_DISCOVERY_PROMPT` asks the LLM: given facts from group A (tool A, query A) and facts from group B (tool B, query B), what typed relations exist between them? Returns `EdgeSuggestion(source_name, target_name, relation)`.

Name→ID resolution is case-insensitive exact match against the group's entity IDs. Edges that fail name resolution are silently dropped (no partial match, no fuzzy matching).

Why LLM for edge discovery rather than keyword overlap? Keyword overlap only finds lexically similar facts. The most valuable edges (e.g. Shannon entropy → black hole thermodynamics Bekenstein–Hawking entropy) have zero keyword overlap but strong conceptual relationship. LLM edge discovery finds these.

7.7 Cross-Topic Edge Discovery

After all expansion rounds, the builder runs one cross-topic pass over the largest-group representative per seed topic. For each group g_a, `cross_topic_sample=3` partner groups from different seeds are sampled without replacement and `_discover_edges(`g_a`, `g_b`, max_edges=3)` is called.

Why this is necessary: Without cross-topic edges, each seed topic's nodes form an isolated clique. The KG before this pass had 7 disconnected components (Louvain modularity 0.780). After: 1 connected component (modularity 0.551). Disconnected topics mean the subgraph sampler cannot produce cross-domain questions; bridging sampling would always fail to connect components.

Why `cross_topic_sample=3` and not all pairs? Full all-pairs over 14 seeds is $\binom{14}{2} = 91$ API calls at max_edges=3 each. With $K = 3$ partners per group, the expected number of cross-topic discovery calls is $14 \times 3/2 = 21$ (each pair counted once). This achieves broad connectivity without a 4× API cost increase.

Why the largest group per seed as representative? Larger groups have more entities, increasing the probability that the LLM finds a valid cross-domain edge. Groups are selected during expansion: the group with the most entity IDs produced for that seed topic is tracked and used.

8 Knowledge Graph Data Structure

File: `src/graph/knowledge_graph.py`

Backed by `networkx.DiGraph`. Entities stored in `_entities: dict[str, Entity]` (id → Entity). Relations stored in `_relations: list[Relation]` (append-only). The DiGraph stores only IDs as node/edge keys; full entity/relation objects live in the parallel dicts.

Why DiGraph rather than MultiDiGraph? A single directed edge per (source, target) pair is sufficient. Multiple edge types between the same pair are represented by the strongest/first added; `add_edge` silently overwrites if the pair already exists. This is intentional: duplicate edges from multiple lookup groups add no structural information and would inflate the neighbour lists used during escalation candidate selection.

Why parallel dict + DiGraph? NetworkX node attributes are accessible via `kg.graph.nodes[id]['name']` but are not typed. The parallel `_entities` dict provides typed, Pydantic-validated entity objects. This avoids the overhead of re-validating on every access and allows IDE type-checking.

Random walk implementation:

```python
def random_walk(self, start_id: str, steps: int) -> list[str]:
    visited = [start_id]; current = start_id
    for _ in range(steps):
        neighbors = list(self.graph.successors(current)) \
                  + list(self.graph.predecessors(current))
        if not neighbors: break
```

```
        current = random.choice(neighbors)
        visited.append(current)
    return visited
```

Walk is on the *undirected* view (successors + predecessors) even though the graph is directed. **Reason:** KG edges are directional by type (causes A→B, supports A→B) but both endpoints are informative regardless of direction. Restricting to successors only would make the walk heavily biased toward high-out-degree hub nodes and fail to reach leaf nodes.

9 Subgraph Sampling

File: `src/graph/subgraph_sampler.py`

Two strategies alternate: even-indexed attempts use random walk, odd-indexed use bridging. The alternation is deterministic by attempt index, not random, to ensure both strategies are used in every batch.

9.1 Random Walk Sampling

```
def sample_by_random_walk(self, min_entities, max_entities, walk_length
    ):
    start = random.choice(entities)
    walked = kg.random_walk(start.id, walk_length)
    # deduplicate preserving order
    unique = list(dict.fromkeys(walked))[:max_entities]
    # pad to min_entities by adding random neighbors if needed
    if len(unique) < min_entities: ...
    return _ensure_connected(unique)
```

walk_length is the primary diameter lever. Longer walks traverse more graph distance before deduplication and trimming. A walk of length 10 on a diameter-12 graph will typically visit nodes with average pairwise distance ∼3–4. A walk of length 30 will visit nodes with average pairwise distance ∼6–8. This is the mechanism behind the complexity scaling.

Why deduplicate preserving order? The walk visits nodes multiple times (it revisits neighbors). Deduplication with order preservation keeps the first-visited (earliest in walk) instances, which are closest to the start node. Later entries in the walk are farther from the start and are trimmed first by the `[:max_entities]` slice, preserving the most locally clustered nodes.

9.2 Bridging Sampling

```
def sample_bridging(self, n_components, entities_per_component):
    seeds = random.sample(entities, n_components)
    for seed in seeds:
        neighbors = kg.get_neighbors(seed.id, hops=2)
        local = [seed.id] + neighbors[:entities_per_component - 1]
        all_ids.extend(local)
    return _ensure_connected(deduplicate(all_ids))
```

get_neighbors uses hops=2 (2-hop BFS), not 1-hop. This ensures each local neighborhood has enough nodes even for low-degree seeds. The `entities_per_component` cap limits how many are included.

Why bridging? Random walk tends to stay in dense local neighborhoods. Bridging forces cross-cluster inclusion by design. The key insight is that cross-domain questions (e.g. connecting CRISPR to black hole thermodynamics via Shannon entropy) require subgraphs that span multiple topic clusters. These cannot arise from random walk on a dense local neighborhood.

9.3 Connectivity Guarantee (`_ensure_connected`)

Both sampling methods can produce disconnected induced subgraphs, especially when:

- Random neighbors are added to meet `min_entities` and they don't lie on the walk path
- Bridging samples from graph regions not yet cross-linked

`_ensure_connected` converts the KG to an undirected view, finds connected components in the induced subgraph via `nx.connected_components`, then for each non-anchor component:

1. Samples up to 3 nodes from the anchor and up to 3 from the component
2. Finds the shortest undirected path between each pair via `nx.shortest_path`
3. Inserts intermediate nodes (path[1:-1]) into the result set

Known issue: noise nodes. At high complexity, `_ensure_connected` inserts KG construction artifacts (`"query executed"`, `"initial lookup"`, `"number of results"`) as connector nodes. These were created during KG building when tool result parsing extracted meta-strings as entity names. They inflate subgraph size without adding reasoning value. Fix: add a blocklist filter in `build_from_seeds` that rejects entities whose names match artifact patterns before they enter the KG.

10 QA Generation from Subgraph

File: `src/graph/subgraph_sampler.py` (`generate_qa_from_subgraph`)

The LLM receives `kg.describe_subgraph(entity_ids)`: a formatted list of all entities with their values and tool sources, followed by all relations between them with evidence strings.

The prompt (`prompts/graph_qa_generator.txt`) instructs the LLM to write a question that:

- Requires connecting at least 3 entities from the subgraph
- Has a definitive, verifiable answer
- Cannot be answered from any single entity alone
- Does not name-drop entities gratuitously (answer must follow from their relationships)

Response schema: `GraphQAResponse(question, answer, reasoning, entities_required)`.

`entities_required` is recorded in `qa.metadata["entities_required"]` for post-hoc analysis but does not affect downstream processing.

11 Uncertainty Injection

File: `src/graph/uncertainty_injector.py`

Six atomic operations, applied sequentially (n=1 in `generate_until_pass.py`, configurable to n=2 via orchestrator):

Operation	Transformation
`entity_merging`	Replace entity name with a description that could match multiple entities
`temporal_displacement`	Add or change temporal constraints (`as of 2019`, `during Cold War`)
`attribute_substitution`	Replace easy attribute with harder one (`capital` → `second-largest city by population`)
`condition_injection`	Add a filtering condition (`among EU member states`, `excluding island nations`)
`aggregation_requirement`	Require combining data across entities (`total` → `average per capita weighted by population`)
`indirection`	Add an intermediate reasoning step (ask about a property of X via Y)

Operations work *within the existing subgraph* — no new entities are added. This is the key distinction from graph escalation (Section 12).

Implementation: each operation calls `complete_json(messages, InjectionResponse)` where `InjectionResponse(question, answer, explanation)` includes the modified question and updated answer. The answer must be updated because some operations change what the correct answer is (e.g. `condition_injection` may narrow the answer set).

`inject_multiple` applies n randomly sampled operations sequentially: `random.sample(ops, min(n, len(ops)))`. Operations are applied to the output of the previous operation (composition). If any operation raises, the partial result is kept.

Why sequential rather than parallel? Each operation's output is the next operation's input. Applying operations to the original QA in parallel and merging would produce contradictory transformations (e.g. two operations that both modify the entity name).

11.1 Adding an Uncertainty Operation

Add to the `OPERATIONS` dict in `uncertainty_injector.py`:

```
OPERATIONS["my_operation"] = (
    "Description of what the LLM should do to the question. "
    "Be specific: what changes, what must be preserved."
)
```

No other changes required. The operation is sampled uniformly with all others during `inject_multiple`. To bias toward specific operations, modify `inject_multiple` to use weighted sampling:

```
weights = [op_weights.get(op, 1.0) for op in ops]
selected = random.choices(ops, weights=weights, k=n_operations)
```

12 Graph Complexity Escalation

File: `src/pipeline/stage2_graph_escalation.py`

12.1 Single Escalation Round

Candidate selection (`_find_extension_candidates`). Candidates are 1-hop neighbors (`_CANDIDATE_HOPS=1`) of the covered entity set, not already in the covered set, sorted by `kg.graph.degree(eid)` descending, capped at `_MAX_CANDIDATES=12`. One random node from the tail (beyond top 11) is added to prevent always picking the same high-degree hub nodes.

Algorithm 2 `GraphComplexityEscalator._escalate_one_round`

```
Require: qa (with source_chunk_ids), kg, max_tool_calls
covered ← {eid ∈ qa.source_chunk_ids ∩ kg entities}
candidates ← _find_extension_candidates(kg, covered) ▷ 1-hop neighbors, degree-sorted, cap 12
Build system prompt: current Q/A, entity descriptions of covered + candidates, tool list
conversation ← [system_msg]
for t = 1 ... max_tool_calls+1 do
    response ← llm.complete(conversation)
    conversation.append({role: assistant, content: response})
    if response matches TOOL_CALL: (\w+) | (.+) then
        result ← registry.execute(tool_name, query=tool_query)
        conversation.append({role: user, content: "OBSERVATION: " + result[:2000]})
    else
        parsed ← _extract_response(response)          ▷ JSON from code block or {.*}
        if parsed then break
        end if
    end if
end for
new_ids ← _resolve_entity_ids(kg, parsed.entities_incorporated)
connector_ids ← _find_connector_nodes(kg, covered, new_ids)
escalated.source_chunk_ids ← covered ∪ new_ids ∪ connector_ids return escalated
```

Why 1-hop only? Guarantees every candidate has at least one direct edge to the existing subgraph. 2-hop candidates may be connected only through an intermediate node not in the subgraph, which would require adding connector nodes and produce unexpected subgraph expansions.

Why degree-sorted? High-degree nodes connect to more of the existing subgraph and more of the graph overall. They produce richer escalation opportunities and are more likely to appear in valid multi-hop chains.

ReAct loop termination. The loop runs up to `max_tool_calls+1` iterations. It terminates early when the LLM emits JSON instead of a `TOOL_CALL` directive. If the loop exhausts without a valid JSON response, a `RuntimeError` is raised and the original QA is kept (escalation failed gracefully).

Connector node insertion (`_find_connector_nodes`). After resolving entity names to IDs, for each new entity that has no direct edge to any covered entity, the shortest undirected path in the full KG is found and intermediate nodes are inserted. This maintains subgraph connectivity invariant.

12.2 Five Escalation Strategies

Strategy	Prompt instruction	Topological effect
`hop_extension`	Add one more reasoning hop: ask about a property of an entity adjacent to a key entity	Linear chain lengthening
`subgraph_bridging`	Connect two currently separate clusters via an intermediate relationship	Highest diameter increase
`multi_entity_aggregation`	Aggregate/compare across all entities matching a condition (rank, find max, compute total)	Width over depth
`constraint_chain`	Chain of filtering constraints, each from a different entity	Sequential narrowing
`inverse_reasoning`	Flip lookup direction: given a property value, find which entity has it	Forces search over retrieval

Strategy selection at runtime (in `generate_until_pass.py`): when `strategy_weights` is not None (i.e. complexity flag is set), the module-level `STRATEGIES` list in `stage2_graph_escalation` is temporarily replaced with a weighted sample drawn via `random.choices`. This is a deliberate monkeypatching approach: the escalator reads `STRATEGIES` from the module namespace at each call, so replacing the list has immediate effect.

Warning: this monkeypatch is not thread-safe. In a multi-process setup, use a different mechanism (e.g. pass weights into `escalate_batch` as a parameter).

12.3 Adding an Escalation Strategy

1. Add the strategy name string to `STRATEGIES` in `stage2_graph_escalation.py`.
2. Add a description of the strategy to `prompts/graph_escalation.txt` in the strategy list section. The LLM chooses which strategy to apply based on this description and its reasoning about the current entities.
3. Add a complexity affinity to `_STRATEGY_AFFINITY` in `generate_until_pass.py` (0.0 = prefer at low cx, 1.0 = prefer at high cx).

The escalator does not enforce which strategy is applied — the LLM selects from the strategy descriptions in the prompt. `parsed.strategy` is the LLM's self-reported choice, recorded in `qa.metadata["strategy"]`.

13 Continuous Complexity Control

File: `scripts/generate_until_pass.py`, `complexity_params()`

13.1 Design: Generative Parameter Scaling vs Post-Hoc Filtering

A post-hoc filter (generate large pool, keep questions above difficulty threshold) has two problems: (1) the filter metric is correlated with the training objective, making the pipeline circular; (2) discarded questions represent wasted API cost proportional to the rejection rate.

Instead, the `-complexity` flag ($c \in [0, 1]$) scales subgraph sampling parameters. When `-complexity` is omitted, `complexity_params(None)` returns the original fixed defaults, and the pipeline is completely unchanged.

13.2 Parameter Scaling

All parameters scale linearly with c:

Parameter	$c = 0$	$c = 1$	Mechanism
`walk_length`	10	30	Primary diameter lever (longer walk = farther nodes)
`walk_min`	2	6	Minimum subgraph size
`walk_max`	5	12	Maximum subgraph size
`bridge_components`	2	4	Number of distant clusters in bridging
`bridge_per_comp`	2	4	Local neighborhood size per cluster
`esc_rounds`	1	1	Fixed (1 round provides marginal gain over 0 rounds at high cost)
`max_tool_calls_qc`	8	12	Advanced solver budget (harder questions need more tool calls)

Strategy weights:

$$w_i(c) = \max(0.05,\ a_i \cdot c + (1 - a_i)(1 - c)) \tag{1}$$

where $a_i \in [0, 1]$ is the affinity of strategy i: `hop_extension`=0.1, `subgraph_bridging`=0.8, `multi_entity_aggregation`=0.7, `constraint_chain`=0.5, `inverse_reasoning`=0.4.

Weights are normalised to sum to 1. The floor of 0.05 prevents any strategy from being completely excluded.

13.3 Why esc_rounds is Fixed at 1

Escalation adds at most 1-hop extensions per round. The topological ceiling is set at sampling time (walk_length, bridge_components). Running 3 escalation rounds triples API cost but yields diminishing returns: the third hop can only extend from whatever the second round added, which can only extend from the first round, which can only extend by 1 hop from the original subgraph. The net diameter increase from 3 rounds is at most 3, but the expected increase is less because the LLM often incorporates entities that are already 1-hop from the original subgraph rather than extending further.

If you want escalation to be a genuine complexity amplifier, the correct fix is: (1) require a minimum post-escalation diameter before accepting the result, (2) pass a complexity target as a constraint to the escalation prompt.

13.4 Empirical Results

Four generation runs, 5 batches of 5 QAs each:

cx	Generated	TOO_EASY	TOO_HARD	PASS
0.10	25	3	11	10 (40%)
0.40	25	0	6	19 (76%)
0.70	25	2	9	13 (52%)
0.95	25	0	6	19 (76%)
Total	100	5	32	61 (61%)

The cx=0.1 pass rate of 40% is almost entirely due to the `lenient_paraphrase` judge (Section 14.2). Without it, the rate was <4%. The cx=0.70 dip relative to cx=0.95 likely reflects a transition zone where questions are complex enough to be hard for the advanced solver but not clearly in `lenient_partial` territory (which starts at $c > 0.55$).

14 Quality Control

Files: `src/pipeline/stage3_quality_control.py`, `src/agents/judge_agent.py`, `src/agents/similarity_`

14.1 Pipeline

1. **Base solver** (`solve_base`): LLM with no tools. Judged *strictly regardless of complexity*. If correct → `TOO_EASY`. Rationale: easy questions must not slip through at any complexity level.
2. **Advanced solver** (`solve_advanced`): ReAct loop with full tool access, up to `max_tool_calls_qc = 8+4c` iterations. Judged with complexity-aware leniency. If incorrect → `TOO_HARD`.
3. **Deduplication**: `SimilarityScorer.is_duplicate` checks cosine similarity of `sentence-transformers/all-` embeddings against all prior PASSes. Threshold: 0.85. If ≥ 0.85 → `DUPLICATE`.
4. **PASS**: `scorer.add_to_index(qa)` to prevent future duplicates. Appended to `passing_qas.jsonl` and `ui_data.json`. Trajectory appended to `passing_trajectories.jsonl`.

14.2 Complexity-Aware Judge

File: `src/agents/judge_agent.py`

```python
def judge(self, reference, predicted, complexity=None) -> bool:
    if complexity is None:
        strictness = "strict"
    elif complexity < 0.45:
        strictness = "lenient_paraphrase"
    elif complexity <= 0.55:
        strictness = "strict"
    else:
        strictness = "lenient_partial"
    # LLM called with strictness injected into judge prompt
```

Three judge modes, each a distinct section in `prompts/judge.txt`:

- **lenient_paraphrase** ($c < 0.45$): accept answer if semantically equivalent to reference, allowing synonym variation, different phrasing, minor additional detail. Rationale: small subgraphs produce obscure specific facts; the solver retrieves the right answer but phrasing diverges from the reference.
- **strict** ($0.45 \leq c \leq 0.55$): require precise factual match. Rationale: mid-range questions are well-formed with clear single-sentence answers where phrasing ambiguity is not the main difficulty.
- **lenient_partial** ($c > 0.55$): accept if the core claim is correct, even if multi-part answer is incomplete. Rationale: multi-hop answers are long; the solver may correctly identify the key relationship but omit supporting detail that doesn't change the answer's correctness.

Base solver is always strict (complexity=None). The leniency only applies to the advanced solver judgment. This asymmetry is intentional: we do not want easy questions to pass because the base solver's wrong answer happened to be lenient-equivalent to the reference.

14.3 Deduplication Implementation

`SimilarityScorer` maintains `_existing_embeddings: list[np.ndarray]`. `is_duplicate` encodes the new question, iterates over all existing embeddings computing cosine similarity, returns True if any exceed threshold. Linear search is $O(n)$ in the number of existing PASSes.

Why linear search instead of FAISS? At 50–200 passing QAs per run, linear search is negligible. FAISS would require an additional dependency and index maintenance. Swap to FAISS if the dataset exceeds ~10k QAs.

Why 0.85 threshold? Empirically chosen. At 0.90, near-paraphrases pass (same question with different entity phrasing). At 0.80, topically related but distinct questions are incorrectly blocked.

15 Trajectory Capture

The advanced solver's ReAct loop records each turn as a `TrajectoryStep`. Steps alternate between assistant turns (thought + optional TOOL_CALL) and user turns (OBSERVATION). The full chain is stored in a `Trajectory` object and saved to `passing_trajectories.jsonl` for every PASS.

Format compatibility: the Tongyi paper (§3.4.2) describes two training modes: ReAct (full interleaved thought/tool/observation) and Context Management (trajectory summary + tool/observation). Both are reconstructable from the stored `Trajectory`: the full sequence is the ReAct format; a summary can be generated from `steps` filtered to `role=assistant` with `tool` populated.

16 Batch Generation Loop

File: `scripts/generate_until_pass.py`

16.1 Async Architecture

All IO-bound operations are `async`. The main loop is `asyncio.run(main())`. Concurrency limits:

- QA generation: `asyncio.Semaphore(8)` (caps concurrent LLM calls during subgraph generation)
- Graph escalation: `max_concurrent=6` (Semaphore in `escalate_batch`)
- QC: `max_concurrent=6` (Semaphore in `run_qc_batch`)

The semaphore caps prevent OpenAI rate-limit errors. At batch_size=8, the pipeline launches 16 concurrent QA generation attempts, of which the first 8 to succeed are kept. Failed attempts (subgraph connectivity failure, LLM error) return None and are excluded.

16.2 No Early Stopping

The loop runs all `max_batches` batches. Early stopping was removed to maximise dataset size per run:

```
# Don't stop early     collect as many passing QAs as possible
# if all_passing: break
```

16.3 Output Files

- `passing_qas.jsonl`: one JSON per line, appended (not overwritten) across runs. Contains full `QAPair.model_dump()`.
- `passing_trajectories.jsonl`: one JSON per line, appended. Contains `Trajectory.model_dump()` for each PASS. May be absent if the QA passed without a trajectory (e.g. orchestrator path).
- `ui_data.json`: single JSON object with `{knowledge_graph, qa_pairs}`. Merged incrementally: existing IDs are skipped, new PASSes appended to `qa_pairs`.

17 Configuration Reference

17.1 `configs/llm_config.yaml`

```yaml
providers:
  openai:
    api_key:  ${OPENAI_API_KEY}
    default_model:  gpt-4o
    # base_url: http://localhost:8000/v1  # uncomment for local vLLM
default_provider:  openai
call_params:
  # Per-agent call params, keyed by params_key
  summary_agent:      {temperature:  0.3, max_tokens:  2000}
  item_writer:        {temperature:  0.7, max_tokens:  4000}
  solver_base:        {temperature:  0.6, max_tokens:  3000}
  solver_advanced:    {temperature:  0.6, max_tokens:  8000}
  judge:              {temperature:  0.1, max_tokens:  1000}
  graph_escalation:   {temperature:  0.7, max_tokens:  4000}
```

Temperature rationale: judge at 0.1 (deterministic, avoids random leniency swings); escalation at 0.7 (creative rewriting benefits from diversity); base solver at 0.6 (moderate sampling for diverse answer formulations).

17.2 `configs/tool_config.yaml`

```yaml
enabled_tools:
    null  # null = all registered tools; or list of names to restrict
search:     {provider:  duckduckgo, max_results:  10}
scholar:    {api_key:  ${SEMANTIC_SCHOLAR_API_KEY}, max_results:  5}
wikipedia:  {max_length:  3000}
visit:      {timeout:  15, max_length:  5000}
table:      {timeout:  15, max_tables:  3, max_rows:  20}
comparison:  {}
```

`enabled_tools` is checked in `PipelineOrchestrator._register_tools`. The `generate_until_pass.py` script does not use this mechanism — it registers tools explicitly.

17.3 `configs/env.sh`

```bash
export OPENAI_API_KEY="..."
export SEMANTIC_SCHOLAR_API_KEY="..."
export BRAVE_API_KEY=""  # optional; search tool falls back to
    DuckDuckGo if empty
```

18 Running and Extending the Pipeline

18.1 Build or Rebuild the KG

```bash
# Full rebuild from all 14 seeds (~30 min, GPT-4o)
python scripts/rebuild_kg.py

# Add new seed topics to existing KG without rebuilding
python scripts/expand_kg.py --seeds "new topic 1" "new topic 2"
```

To add new seed topics: add them to `data/seeds/seed_entities.json` and run `expand_kg.py`. The cross-topic pass is re-run after expansion to link new nodes to existing topics. Existing KG nodes are not modified.

18.2 Generate QA Pairs

```
# Original pipeline (no complexity scaling)
python scripts/generate_until_pass.py --batches 5 --batch-size 8

# With complexity control
python scripts/generate_until_pass.py --complexity 0.7 --batches 5 --
    batch-size 8
```

18.3 Specialising the Pipeline for a Domain

To generate QAs in a specific domain (e.g. biomedical):

1. Replace `data/seeds/seed_entities.json` with domain-specific seeds.
2. Optionally restrict `tool_config.yaml enabled_tools` to domain-appropriate retrievers (e.g. `scholar` only for academic domains).
3. Rebuild the KG.
4. Adjust uncertainty operations: disable operations that don't make sense for the domain (e.g. `temporal_displacement` is less useful for mathematical topics).
5. Adjust judge prompt (`prompts/judge.txt`) to account for domain-specific answer formats (e.g. chemical formula equivalences, numeric tolerances).

18.4 Modifying Prompts

All prompts are plain text files with Python `.format(**kwargs)` substitution. Variable names are the `{...}` placeholders in the file. Each prompt file is loaded once at agent/escalator construction time.

Prompt files and their variables:

- `graph_qa_generator.txt`: `{subgraph_description}`
- `graph_escalation.txt`: `{question, answer, current_entities, extension_candidates, tool_descriptions, max_tool_calls}`
- `uncertainty_injection.txt`: `{operation_name, operation_description, question, answer, kg_context}`
- `judge.txt`: `{reference, predicted, strictness}`
- `summary_agent.txt`: `{text}`
- `item_writer_seed.txt` / `item_writer_escalate.txt`: item writer prompts (Track A)

19 Visualiser

19.1 Server (`ui/serve.py`)

Extends `http.server.SimpleHTTPRequestHandler`. Two additional endpoints:

- `POST /api/generate`: JSON body `{complexity, batches, batch_size, api_key}`. If `api_key` is provided, it is written to an environment variable before spawning the subprocess (takes priority over `env.sh`). Spawns `generate_until_pass.py` as a subprocess. Thread-safe job state via `threading.Lock`.
- `GET /api/status`: Returns `{running: bool, complexity: float, elapsed_s: float, log_tail: list[str]}` where `log_tail` is the last 20 lines of subprocess stdout.

`GET` requests to `/data/output/*` are served as static files by the base handler.

19.2 Viewer (`ui/viewer.html`)

D3.js force-directed graph. Data loaded from `/data/output/ui_data.json`. Polling: `GET /api/status` every 2.5s while job is running. Auto-reload of `ui_data.json` 1.2s after job completion.

QA list: filter tabs (All / Pass / Too Easy / Too Hard), per-card complexity badge (color-coded 0.00–1.00), verdict badge, strategy badge, uncertainty operation badge. Detail panel: Q&A tab, solver comparison tab (base vs. advanced answers), escalation chain tab (question text at each history step).

KG view: subgraph nodes highlighted on QA selection, color-coded by tool source, zoom/-pan/drag, node click popups.

20 Known Issues and Fix Locations

Noise nodes in high-cx subgraphs. KG construction artifacts ("query executed", "number of results") enter the graph as entity nodes when tool result parsing is too permissive. Fix: add a name blocklist filter in `GraphBuilder._lookup_and_atomise` before `kg.add_entity`.

Strategy monkeypatch is not thread-safe. `generate_until_pass.py` temporarily replaces the module-level `STRATEGIES` list. Fix: pass `strategy_weights` as a parameter to `escalate_batch` and `_escalate_one_round`, and use it in weighted strategy selection there.

Deduplication is O(n) in PASS count. `SimilarityScorer.is_duplicate` iterates all existing embeddings. Above ∼10k PASSes, use FAISS: replace the list with a `faiss.IndexFlatIP` index.

DuckDuckGo rate limiting. The search tool retries 3× with 3s backoff. If DuckDuckGo is consistently rate-limiting, set a `BRAVE_API_KEY` in `env.sh` to use Brave Search instead.

Entity name resolution in escalation is exact-match. `_resolve_entity_ids` requires case-insensitive exact match on entity name. LLM-generated `entities_incorporated` lists often contain paraphrased names that fail resolution. Fix: add fuzzy string matching (e.g. `rapidfuzz.process.extractOne`) with a similarity threshold.

Trajectory capture only in generate_until_pass.py path. The `PipelineOrchestrator.run_graph_base` path does not capture trajectories (the `QuestionSolverAgent.solve_advanced` return value is not threaded through QC to the output writer). Fix: modify `QualityController.evaluate_single` to return the trajectory alongside the verdict.

References

[1] Qiao et al. WebFrontier: Synthesizing Agent Training Data via Iterative Complexity Escalation. *arXiv:2509.13309v2*, 2025.

[2] Anonymous. Tongyi DeepResearch: Scaling Agentic AI via Graph-Grounded Open-World Data Synthesis. *arXiv:2510.24701v2*, 2025.

[3] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*.